TJ_modul_6

Nama : Helven Marcelyno Aldy Dwy Anggara

Nim : 108092500012

Kelas : IT-08-02

FILE CODE

1. DLL.h

DLL.h | DLL.cpp | main.cpp

```
#ifndef DLL_H_INCLUDED
#define DLL_H_INCLUDED

#include <iostream>
#include <string>

using namespace std;

struct history {
    string title;
    string url;
    string timestamp;
    int visitcount;
};

typedef history infotype;
typedef struct elmList *address;

struct elmList {
    infotype info;
    address next;
    address prev;
};

struct List {
    address First;
    address Last;
};

void createList(List &L);
address createElemen(infotype dataBaru);
void insertLast(List &L, address P);
void insertAfter(address Prec, address P);
void deleteFirst(List &L, address &P);
void deleteLast(List &L, address &P);
void deleteAfter(address Prec, address &P);
void printList(List L);

void insertFirst(List &L, address P);
void insertAscending(List &L, address P);
bool findElemen(List L, string X);
int moreThanOnce(List L);

#endif

```

2. DLL.cpp

DLL.h | DLL.cpp | main.cpp

```cpp
#include "DLL.h"

void createList(List &L) {
    L.First = NULL;
    L.Last = NULL;
}

address createElemen(infotype dataBaru) {
    address P = new elmList;
    if (P != NULL) {
        P->info = dataBaru;
        P->next = NULL;
        P->prev = NULL;
    }
    return P;
}

void insertLast(List &L, address P) {
    if (L.First == NULL) {
        L.First = P;
        L.Last = P;
    } else {
        P->prev = L.Last;
        L.Last->next = P;
        L.Last = P;
    }
}

void insertAfter(address Prec, address P) {
    if (Prec != NULL) {
        P->next = Prec->next;
        if (Prec->next != NULL) {
            Prec->next->prev = P;
        }
        Prec->next = P;
        P->prev = Prec;
    }
}

void deleteFirst(List &L, address &P) {
    P = L.First;
    if (P != NULL) {
        if (L.First->next == NULL) {
            L.First = NULL;
            L.Last = NULL;
        } else {
            L.First = L.First->next;
            L.First->prev = NULL;
            P->next = NULL;
        }
```

DLL.h | DLL.cpp | main.cpp

```
            P->next = NULL;
        }
    }
}

void deleteLast(List &L, address &P) {
    P = L.Last;
    if (P != NULL) {
        if (L.First == L.Last) {
            L.First = NULL;
            L.Last = NULL;
        } else {
            L.Last = L.Last->prev;
            L.Last->next = NULL;
            P->prev = NULL;
        }
    }
}

void deleteAfter(address Prec, address &P) {
    if (Prec != NULL && Prec->next != NULL) {
        P = Prec->next;
        Prec->next = P->next;
        if (P->next != NULL) {
            P->next->prev = Prec;
        }
        P->next = NULL;
        P->prev = NULL;
    }
}

void printList(List L) {
    address P = L.First;
    while (P != NULL) {
        cout << P->info.title << " " << P->info.url << " " << P->info.timestamp << " " << P->info.visitcount << endl;
        P = P->next;
    }
    cout << endl;
}

void insertFirst(List &L, address P) {
    if (L.First == NULL) {
        L.First = P;
        L.Last = P;
    } else {
        P->next = L.First;
        L.First->prev = P;
        L.First = P;
    }
}

void insertAscending(List &L, address P) {
    if (L.First == NULL) {
        insertFirst(L, P);
    } else if (P->info.visitcount <= L.First->info.visitcount) {
        insertFirst(L, P);
    } else {
        address Prec = L.First;
        while (Prec->next != NULL && Prec->next->info.visitcount <= P->info.visitcount) {
            Prec = Prec->next;
        }
        if (Prec->next == NULL) {
            insertLast(L, P);
        } else {
            insertAfter(Prec, P);
        }
    }
}

bool findElemen(List L, string X) {
    address P = L.First;
    while (P != NULL) {
        if (P->info.title == X) {
            return true;
        }
        P = P->next;
    }
    return false;
}

int moreThanOnce(List L) {
    int count = 0;
    address P = L.First;
    while (P != NULL) {
        if (P->info.visitcount > 1) {
            count++;
        }
        P = P->next;
    }
    return count;
}
```

Main.cpp

SLL.h SLL.cpp main.cpp

```cpp
#include "SLL.h"

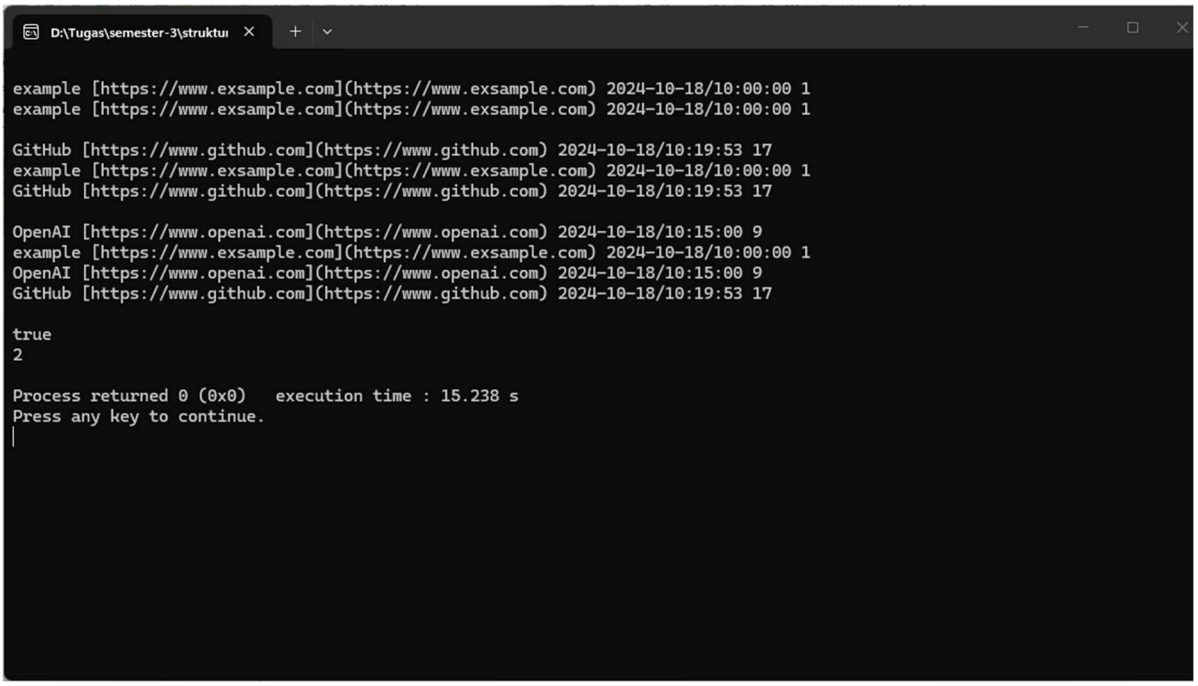
int main() {
    List L;
    createNewList(L);
    int pilihan;

    do {
        cout << "\nMenu Restoran Pagi Sore:" << endl;
        cout << "1. Tambah data masakan" << endl;
        cout << "2. Tampilkan semua data masakan" << endl;
        cout << "3. Tampilkan semua data masakan yang masih tersedia" << endl;
        cout << "4. Tampilkan semua data masakan yang paling laku terjual (best seller)" << e
        cout << "5. Hapus semua data masakan yang sudah laku terjual" << endl;
        cout << "6. Transaksi pembelian masakan (Bonus)" << endl;
        cout << "0. Selesai" << endl;
        cout << "Pilih menu : ";
        cin >> pilihan;

        switch (pilihan) {
            case 1: {
                int n;
                cout << "Masukkan jumlah data masakan : ";
                cin >> n;
                addNElementList(L, n);
                break;
            }
            case 2:
                showAllFood(L);
                break;
            case 3:
                showAvailableFood(L);
                break;
            case 4:
                showBestSeller(L);
                break;
            case 5:
                deleteAllSoldOut(L);
                break;
            case 6: {
                int no, porsi;
                cout << "Nomor Masakan yang dibeli : ";
                cin >> no;
                cout << "Jumlah porsi yang dibeli : ";
                cin >> porsi;
                transaction(L, no, porsi);
                break;
            }
            case 0:
                cout << "Program selesai." << endl;
                break;
            default:
                cout << "Pilihan tidak valid!" << endl;
        }
    } while (pilihan != 0);

    return 0;
}
```

BUILD & RUN


D:\Tugas\semester-3\struktur
example https://www.exsample.com 2024-10-18/10:00:00 1
example https://www.exsample.com 2024-10-18/10:00:00 1
GitHub https://www.github.com 2024-10-18/10:19:53 17
example https://www.exsample.com 2024-10-18/10:00:00 1
GitHub https://www.github.com 2024-10-18/10:19:53 17
OpenAI https://www.openai.com 2024-10-18/10:15:00 9
example https://www.exsample.com 2024-10-18/10:00:00 1
OpenAI https://www.openai.com 2024-10-18/10:15:00 9
GitHub https://www.github.com 2024-10-18/10:19:53 17
true
2
Process returned 0 (0x0) execution time : 15.238 s
Press any key to continue.